
Project Title: Password Manager in C++ with Encrypted Storage

Team Members:

Kaushal N – 2024503555

Krishna Kailash Yedida – 2024503575

Sanjay C – 2024503507

Rangesh – 202403549

Objective:

To develop a simple command-line based password manager in C++ that securely stores, retrieves, modifies, and deletes user credentials such as app name, username, and password using basic encryption techniques. The program is protected by a master password to ensure that only authorized users can access the stored data.

Project Description:

This project is built around a single class named Password which handles all major functionalities including encryption, decryption, file input/output, and credential management. User data is encrypted using the master password before being stored in a local file, and decrypted only when access is granted through correct authentication.

Each credential entry consists of the application name, username, and password. These entries are encrypted using a substitution cipher method and stored line-by-line in a text file. The master password, used for all encryption and decryption operations, is itself encrypted using a fixed salt and stored securely.

Key Features:

- **Master Password Authentication:**
The user is prompted to create a master password during the first run. This password is then encrypted and stored in the first line of the storage file. In future sessions, the user must re-enter the master password to decrypt and access any saved data.
- **Encryption and Decryption of Credentials:**
Each field (app name, username, and password) is encrypted using a character-wise addition with the master password. Decryption uses a reverse operation to reconstruct the original input values.
- **File Handling:**
The application uses a file named passwords.txt for persistent storage. The first line

contains the encrypted master password. Each subsequent line represents a stored set of credentials, with encrypted fields separated by an asterisk symbol.

- **Supported Operations:**
 - **Add New Entry:** Input and store a new set of credentials after encrypting them.
 - **Read Entry:** Decrypt and display credentials for a specific application.
 - **Delete Entry:** Locate and remove a stored entry after matching the app name.
 - **Modify Entry:** Delete the existing entry and allow the user to input and save updated data.
-

Working Overview:

1. **Startup and Master Authentication:**

When the program starts, it checks if a master password is already stored. If not, it prompts the user to create one. The user must then correctly enter this master password each time they run the program.
 2. **Main Menu and Functional Flow:**

Upon successful authentication, the user can select from multiple operations such as adding, retrieving, deleting, or modifying stored credentials. Each operation interacts with the encrypted data file and handles secure input and output accordingly.
 3. **Encryption Logic:**

The encryption process uses the master password as a key to modify each character in the input fields by adding corresponding characters and wrapping around using modulo 256 arithmetic. This ensures that the data is not stored in plain text and is unreadable without the correct master password.
-

Conclusion:

This C++ based password manager effectively demonstrates the application of object-oriented programming, file management, and basic encryption. It provides users with a simple, offline tool to store and manage their login credentials securely using text file storage with custom encryption logic.
